

Ускорение Unet Inference

Авторы

Тюменцева Ирина

Никитюк Илья

Потапенко Антон



Содержание

01 Baseline

02 Torch compile

03 TVM

04 Quantization

05 Pruning

start

01 Baseline

Baseline

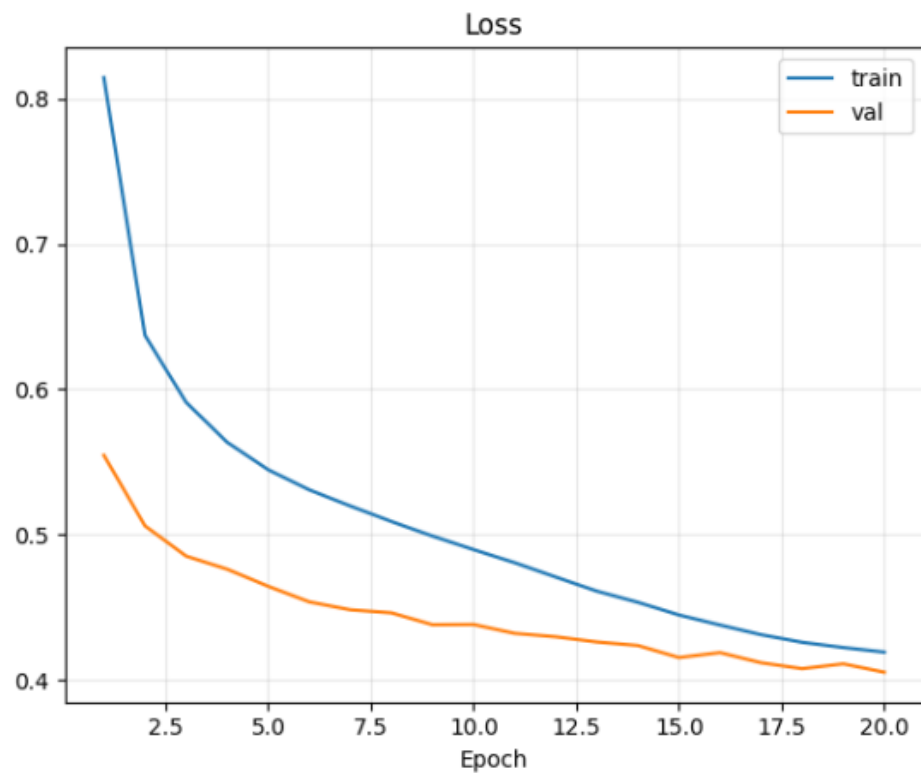
Task parameters

- **Encoder:** ResNet101
- **Pretrain encoder dataset:** ImageNet
- **Model parameters:** 51 524 833
- **Train decoder dataset:** COCO
- **Dataset size:** ~ 123 000 images
- **Number of classes:** 81
- **dtype:** bfloat16

Model parameters

- **Loss:** CrossEntropyLoss
- **Optimizer:** AdamW
- **Learning rate:** $10e-3$ + scheduler
- **Epochs:** 20

Loss function



Main metric

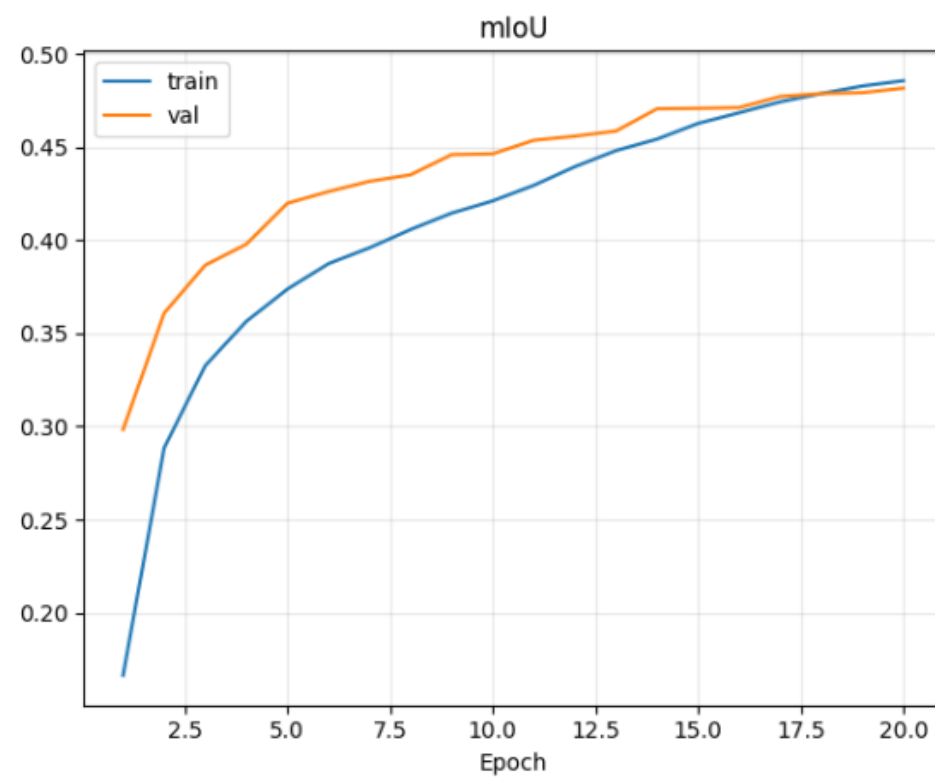
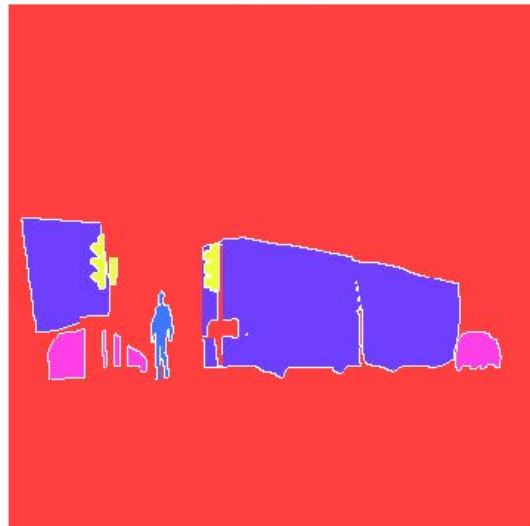


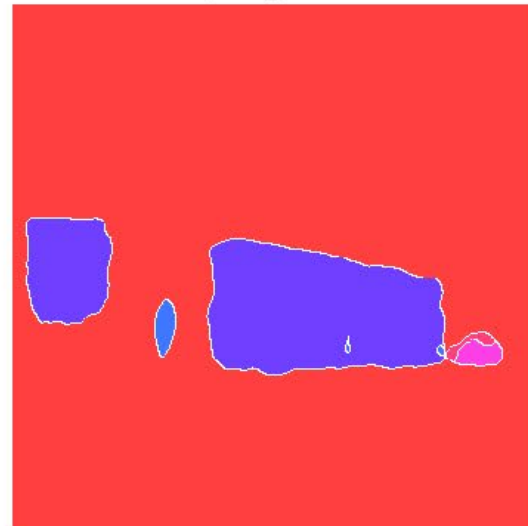
image #2882



true_mask_val



pred_mask



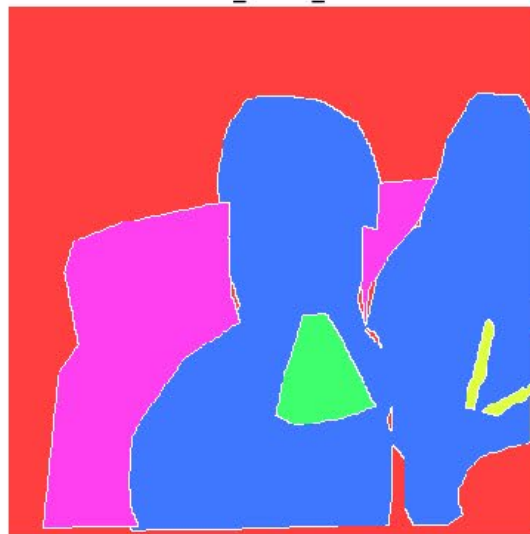
classes in sample

- 0: background
- 6: bus
- 3: car
- 1: person
- 10: traffic light
- 8: truck

image #512



true_mask_val

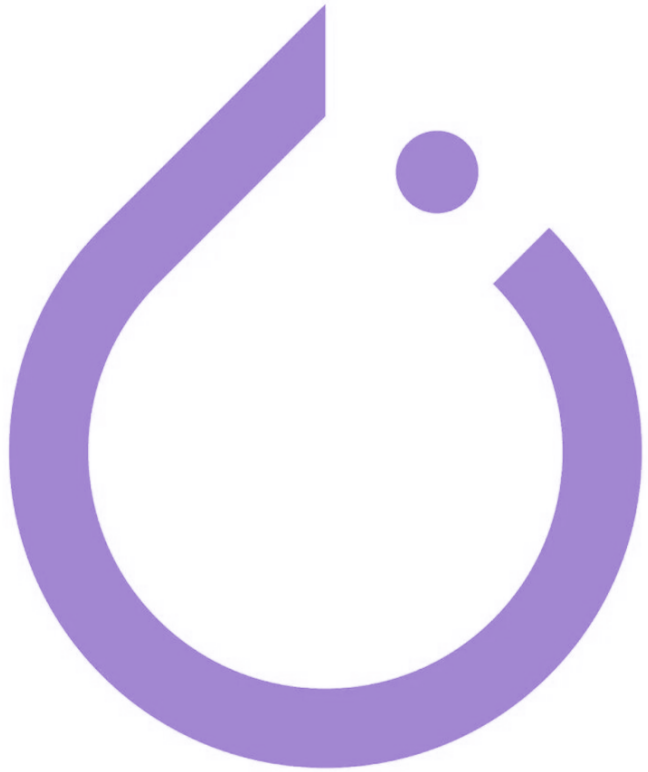


pred_mask



classes in sample

- 1: person
- 0: background
- 58: couch
- 54: pizza
- 57: chair
- 60: bed
- 44: knife
- 29: suitcase
- 61: dining table



02 Torch Compile

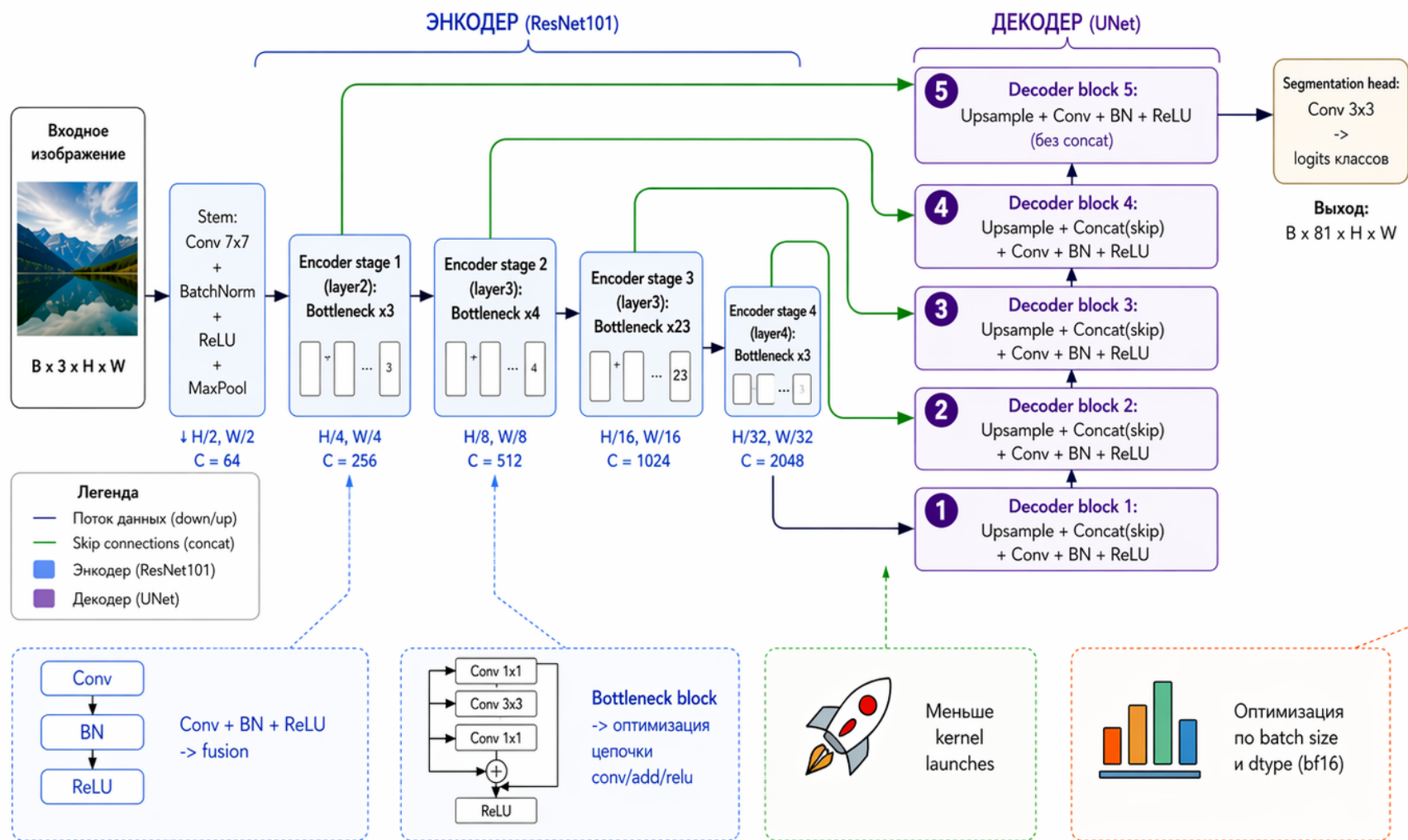
Torch.compile

Что было сделано

- baseline модель FP16/BF16 и сравниваем с torch.compile
- torch.compile запущена в 2 режимах:
 - *default*
 - *max-autotune*
- Были оценены основные параметры inference:
 - *Latency*
 - *throughput with batch sizes: 1, 2, 4, 8, 16, 32, 64*
- Качество модели было оценено метрикой *mIoU*

Схема вычислений модели, которую оптимизирует torch.compile

UNet + ResNet101 encoder для сегментации

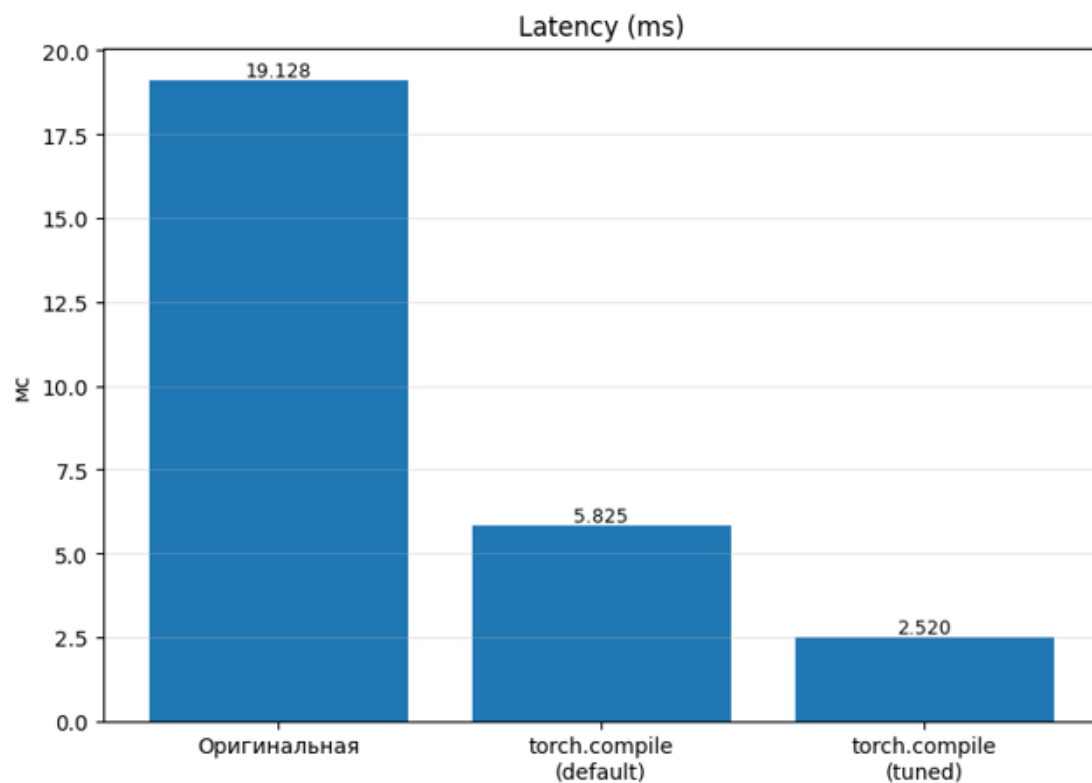


Что делает torch.compile

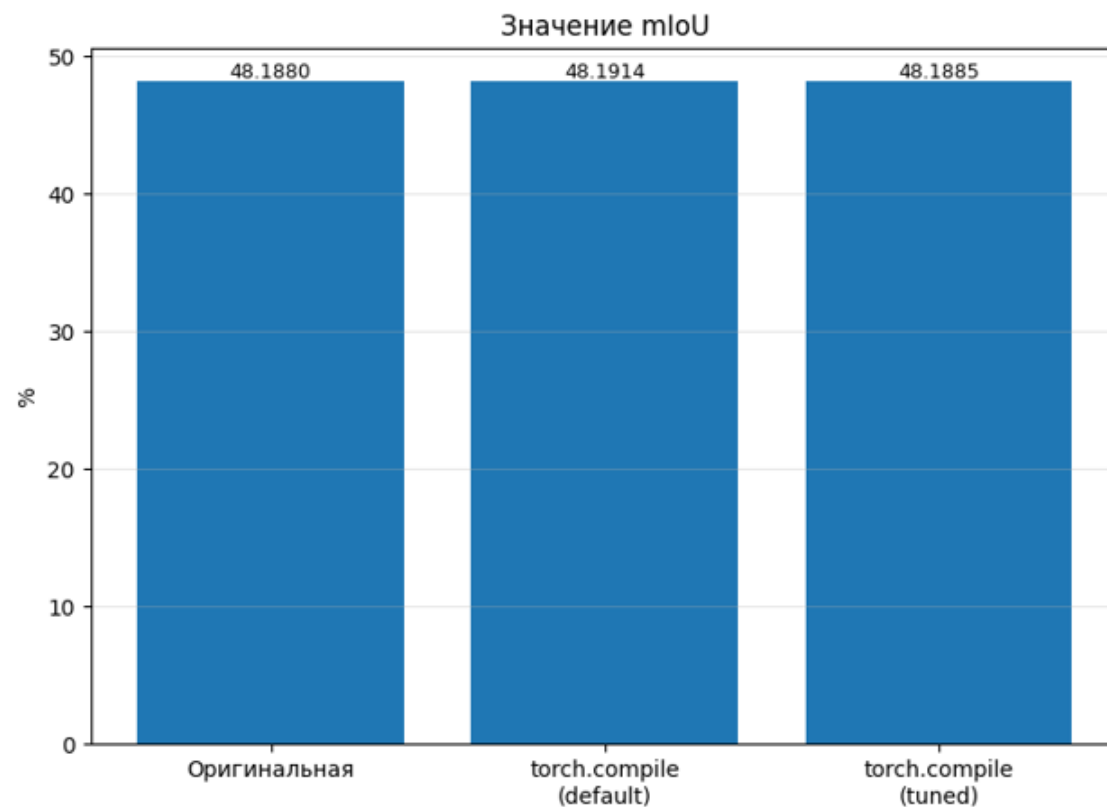
-  1. Захватывает tensor-операции в граф
-  2. Убирает Python overhead
-  3. Сликает типовые последовательности операций
-  4. Генерирует более эффективное исполнение на GPU

Оптимизируются захваченные фрагменты графа; между ними возможны graph breaks

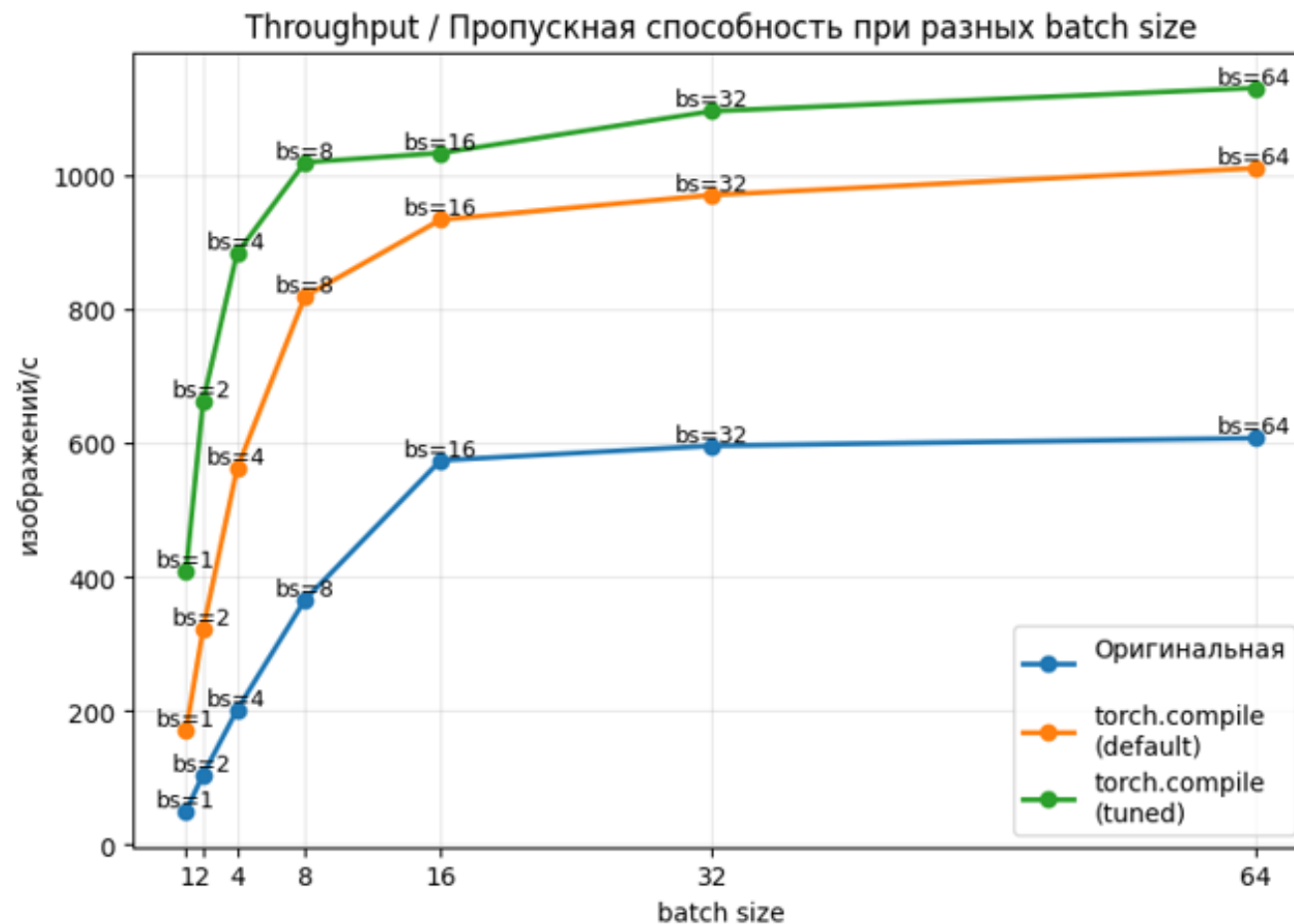
Latency



Main metric



Baseline & Torch compile Throughput





03

Open Machine Learning Compiler Framework (TVM)

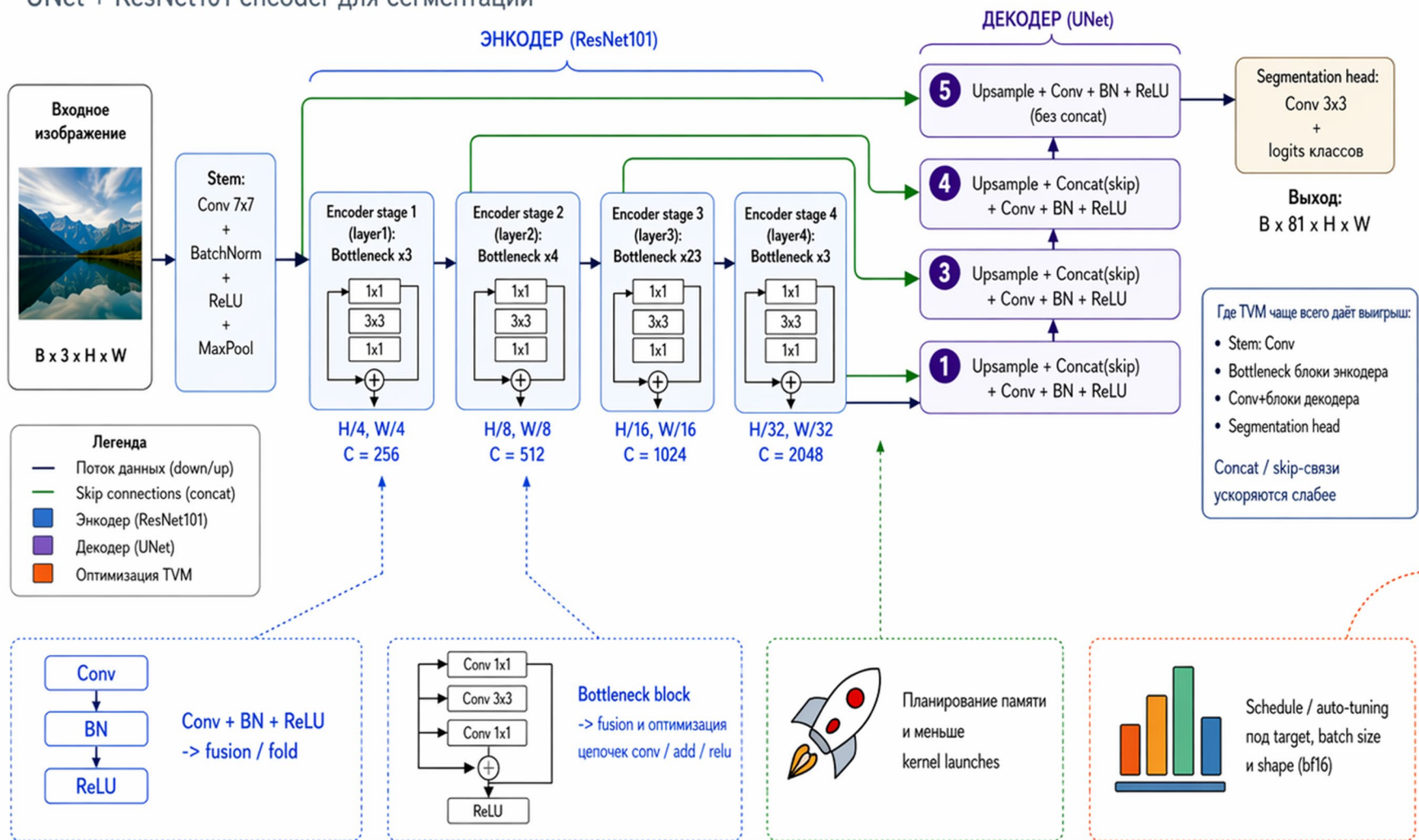
TVM

Что было сделано

- baseline модель в BF16 и сравниваем с TVM
- Pipeline:
 - *torch.export -> TVM Relax -> CUDA build -> VM runtime*
- Были оценены основные параметры inference:
 - *Latency*
 - *throughput with batch sizes: 1, 2, 4, 8, 16, 32, 64*
- Качество модели было оценено метрикой *mIoU*

Схема вычислений модели, которую оптимизирует TVM

UNet + ResNet101 encoder для сегментации

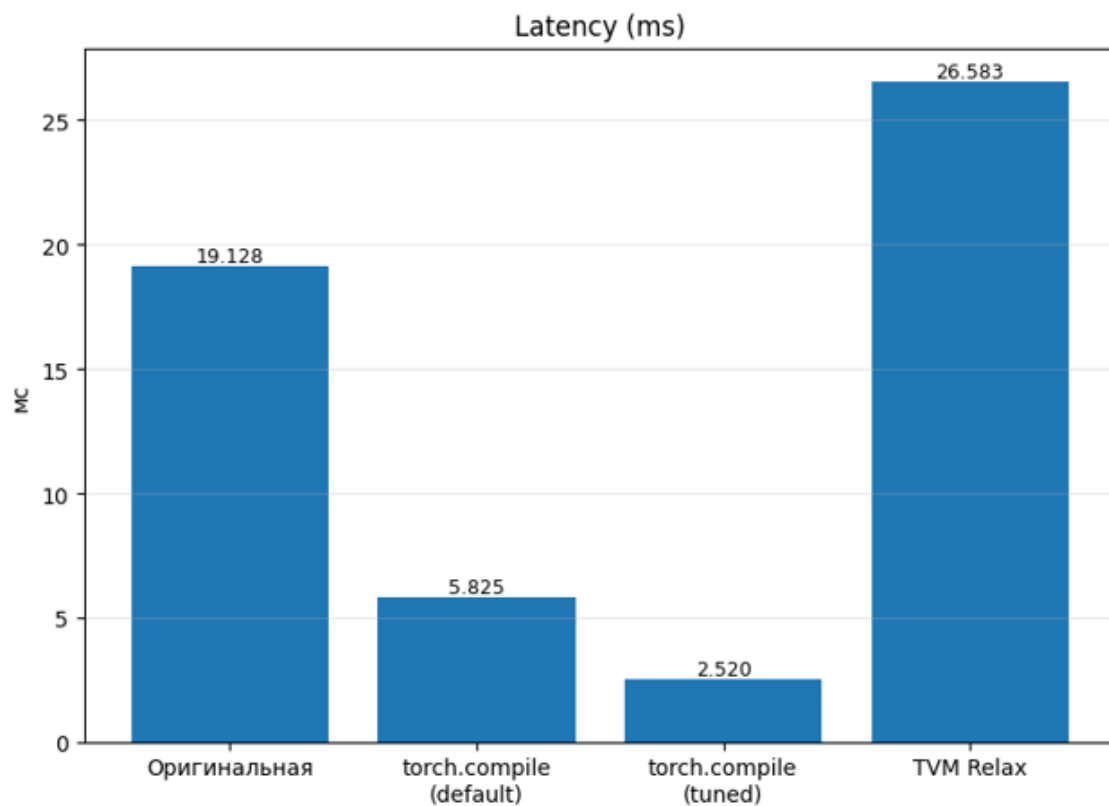


Что делает TVM

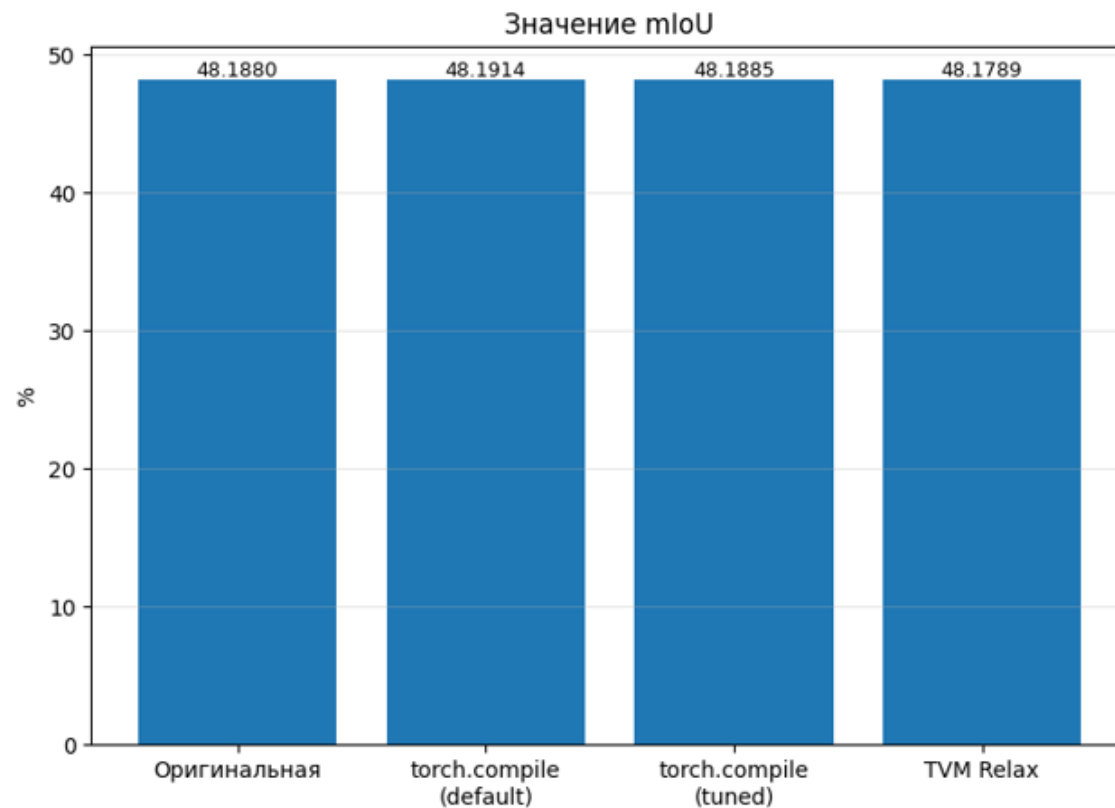
- 1 Строит и понижает граф в Relay / TIR
- 2 Делает graph-level оптимизации и fusion
- 3 Подбирает schedule и генерирует CUDA kernels
- 4 Ускоряет inference на GPU: меньше latency, выше throughput

Оптимизируются прежде всего conv-heavy фрагменты графа; элементы управления — в делегатах и пользовательских conv/add/relu.

Latency

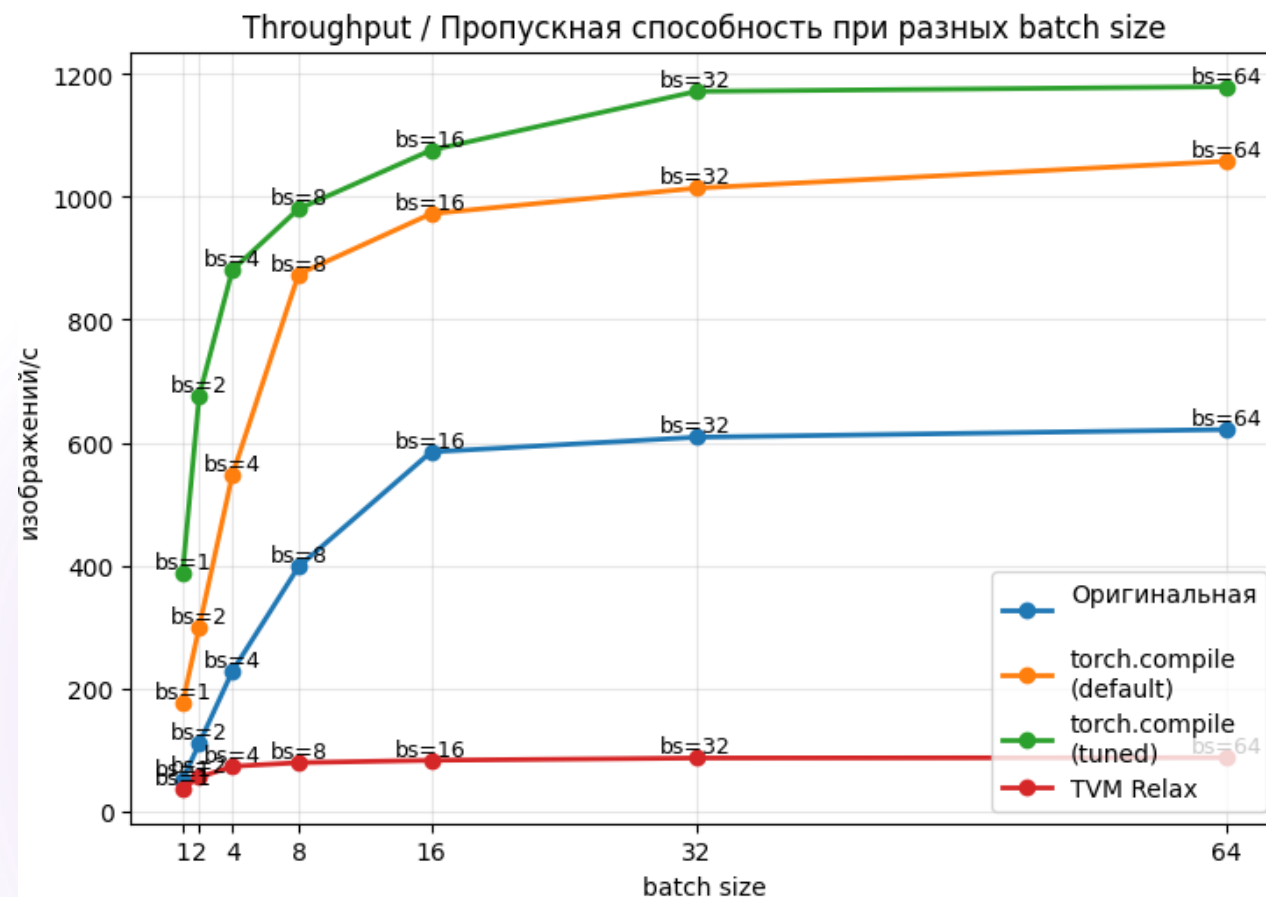


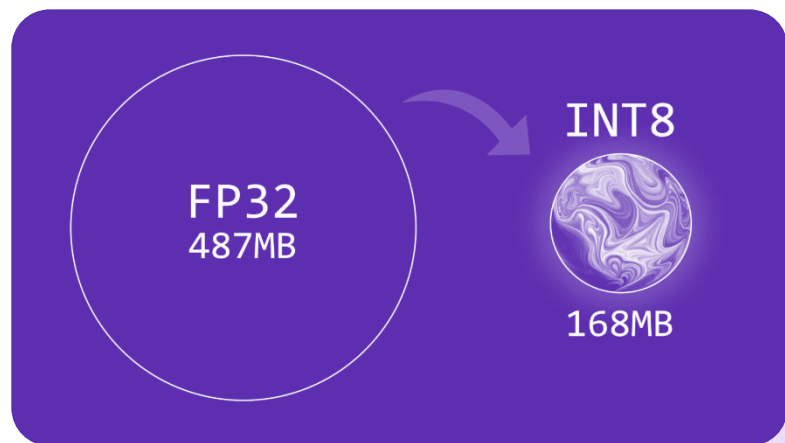
Main metric



Baseline & Torch compile & TVM

Throughput

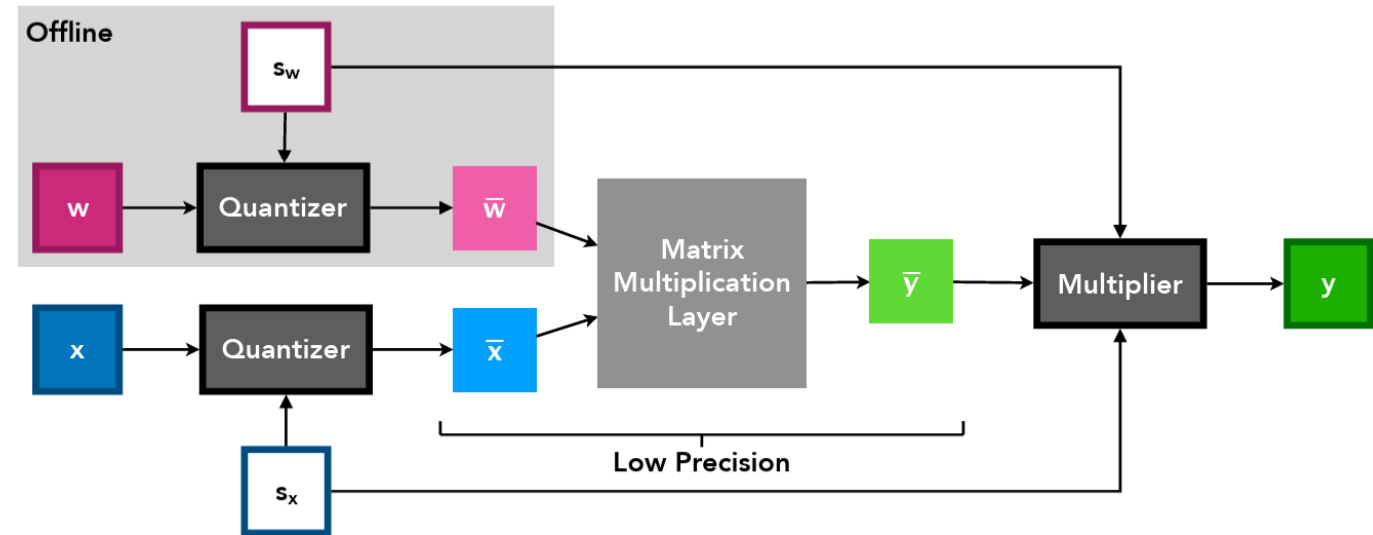




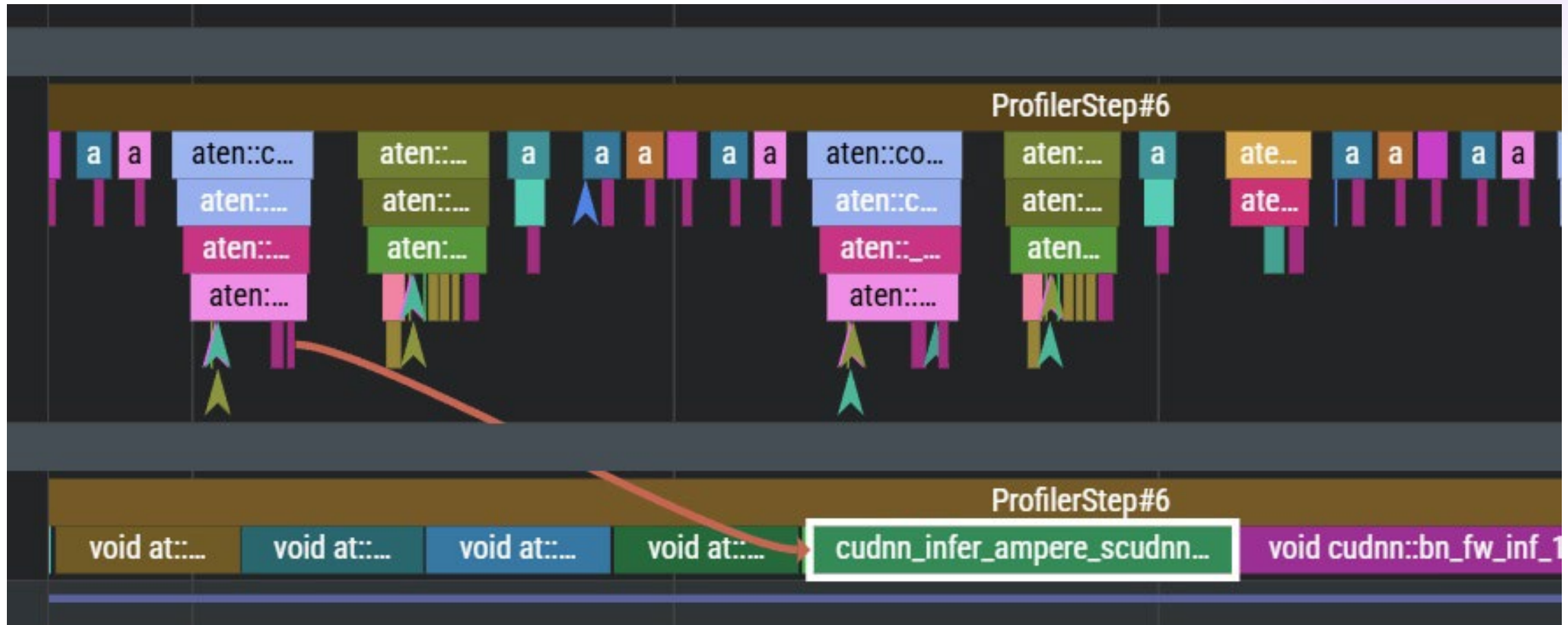
04 Quantization

Learned Step Size Quantization

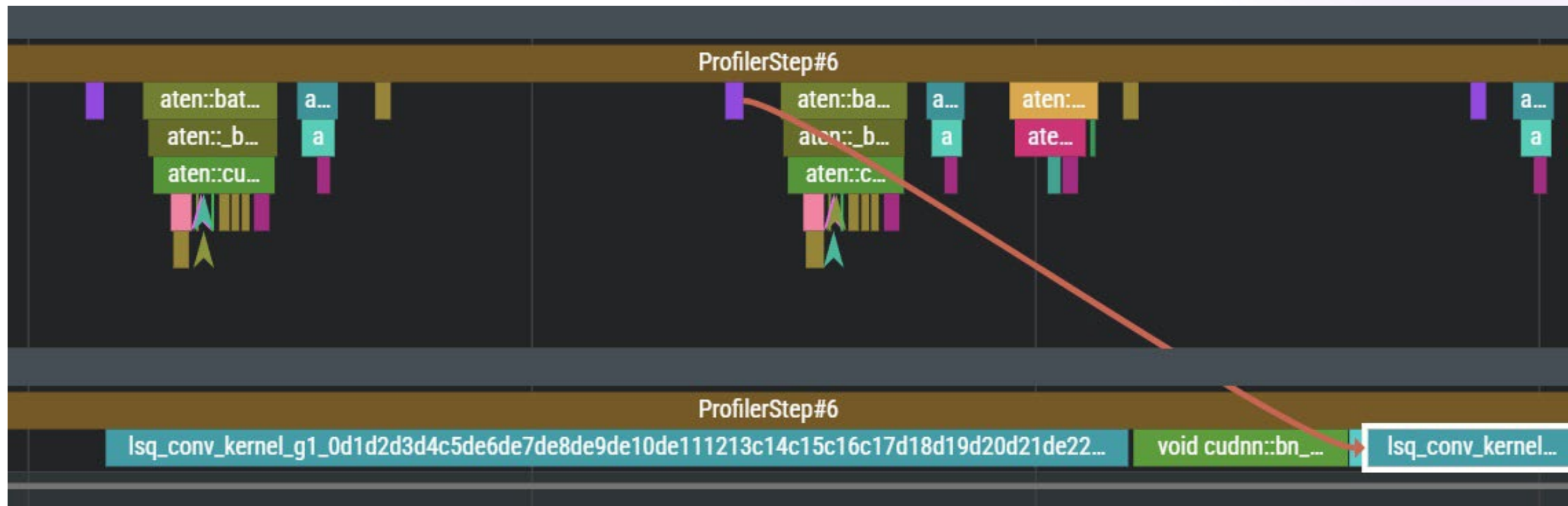
- Обучение свёрточных слоёв в 8 bit
- Инференс с `torch.nn.functional.conv2d`
- Инференс с Triton-ядром



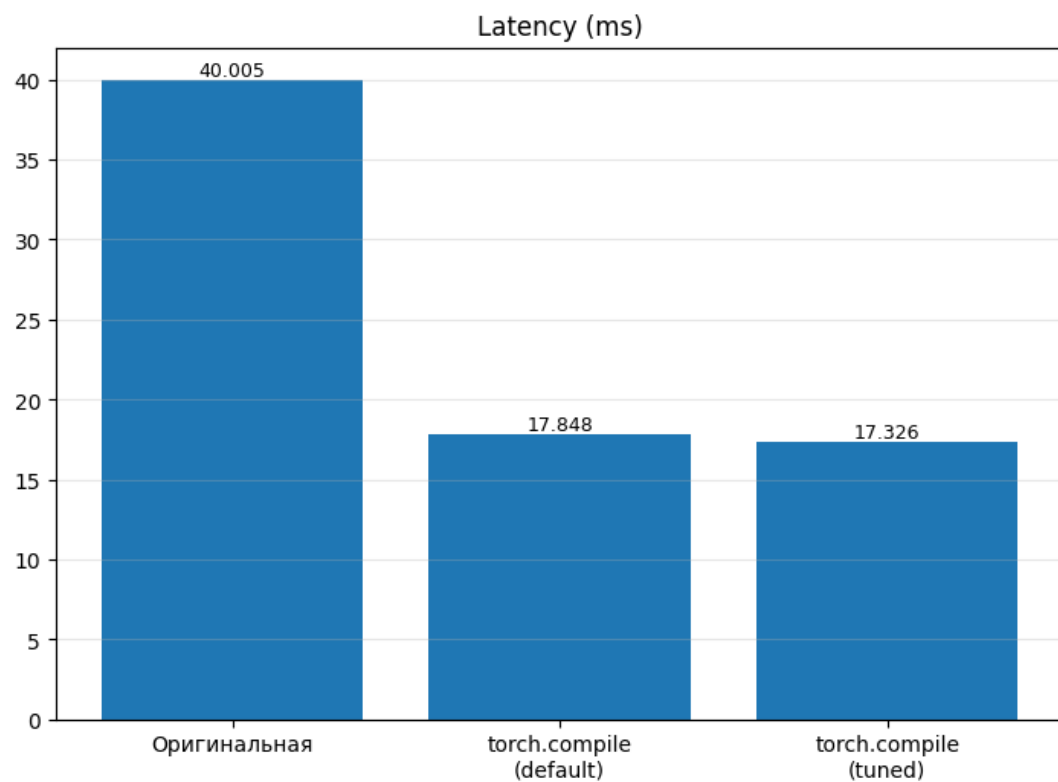
Inference c F.conv2 (fp16)



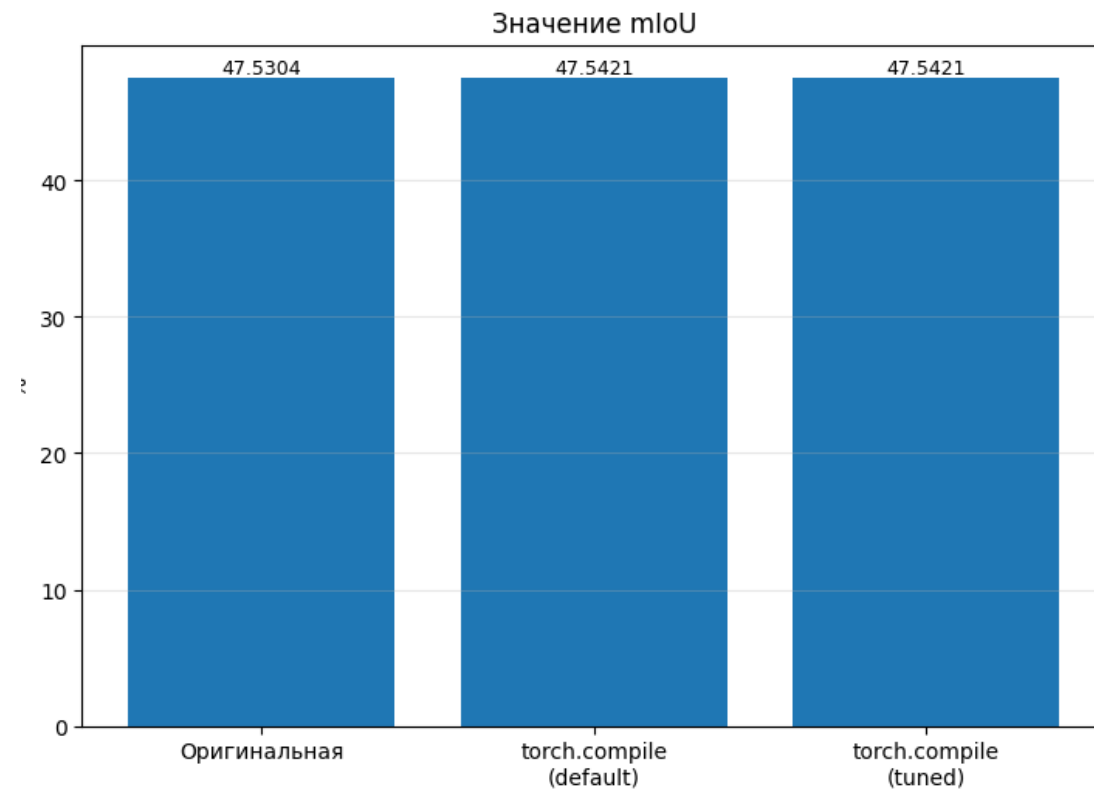
Inference Triton-kernel (int8)



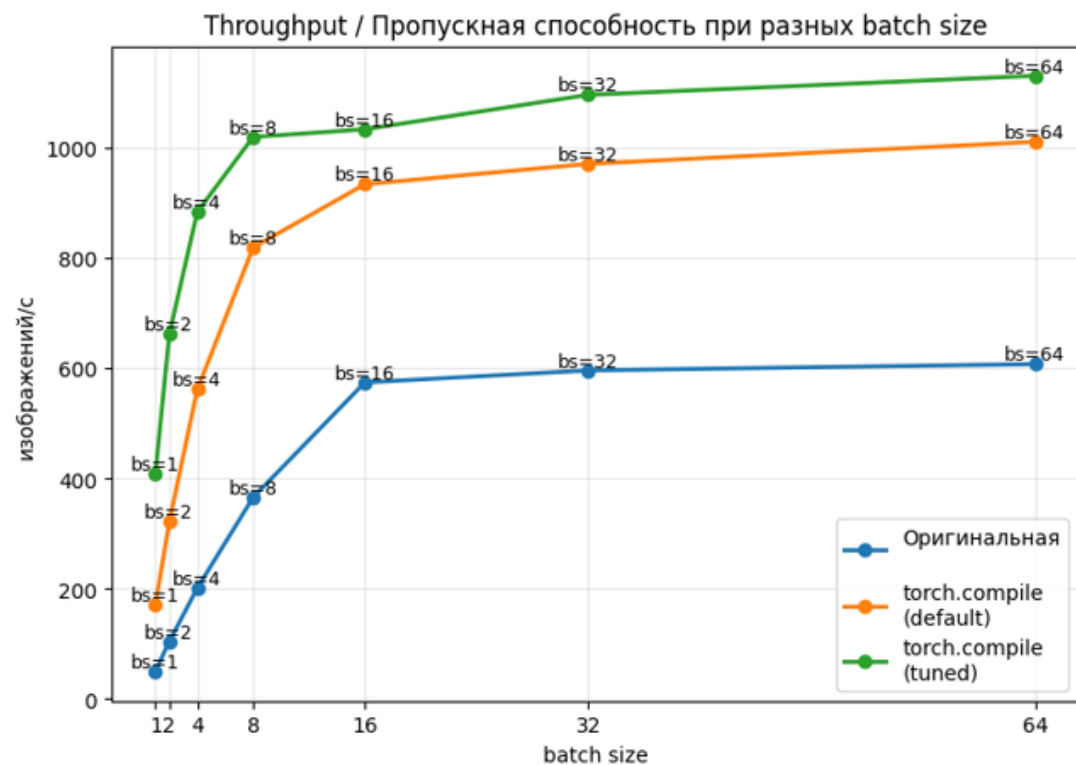
Latency



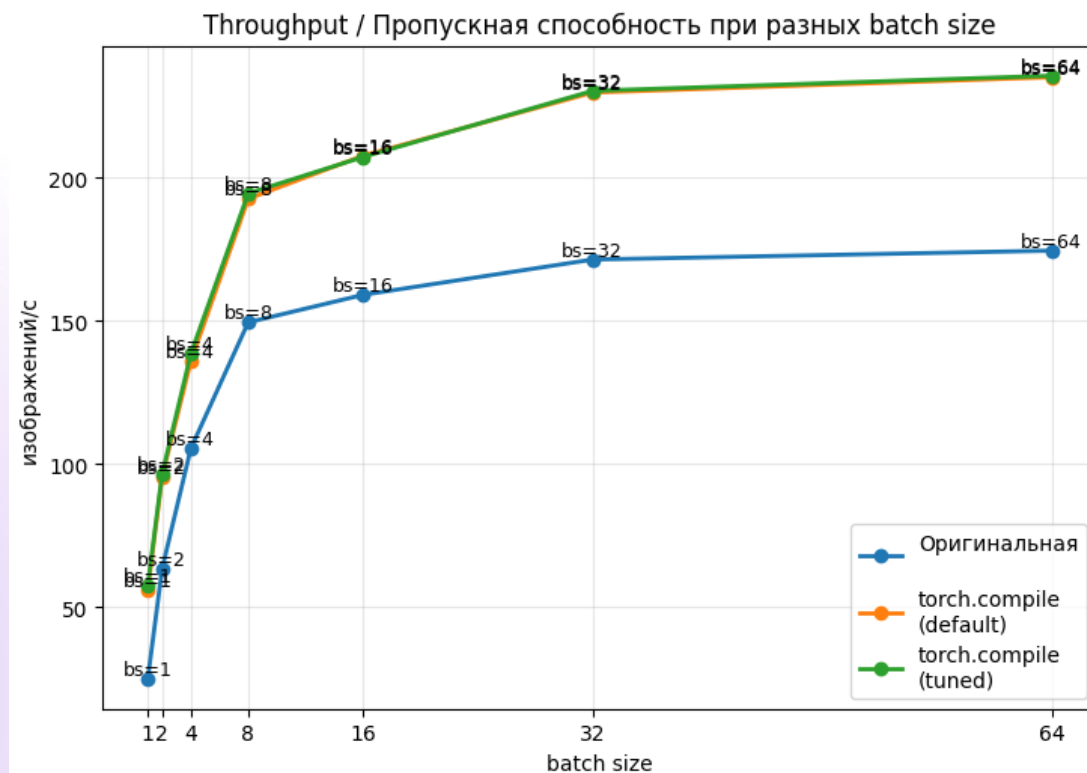
Main metric

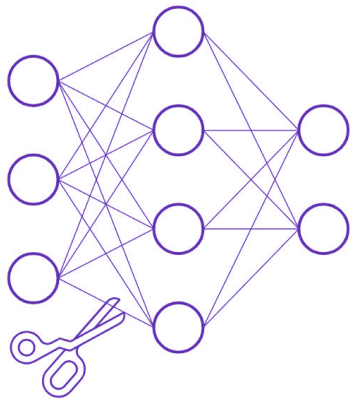


Throughput

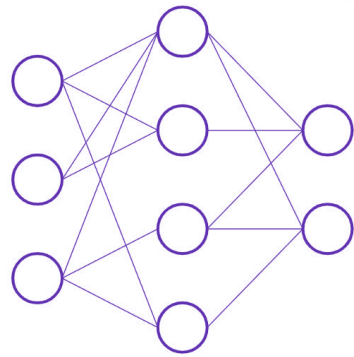


Throughput with lsq





Before pruning

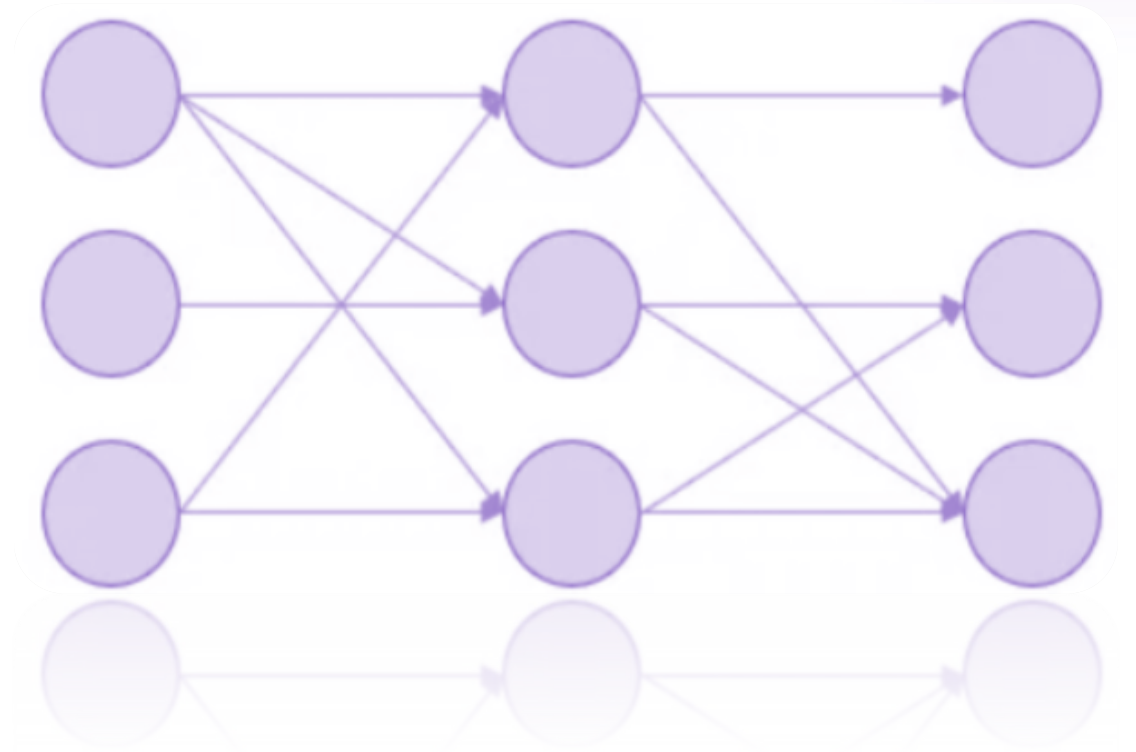


After pruning

05 Pruning

Unstructured Pruning

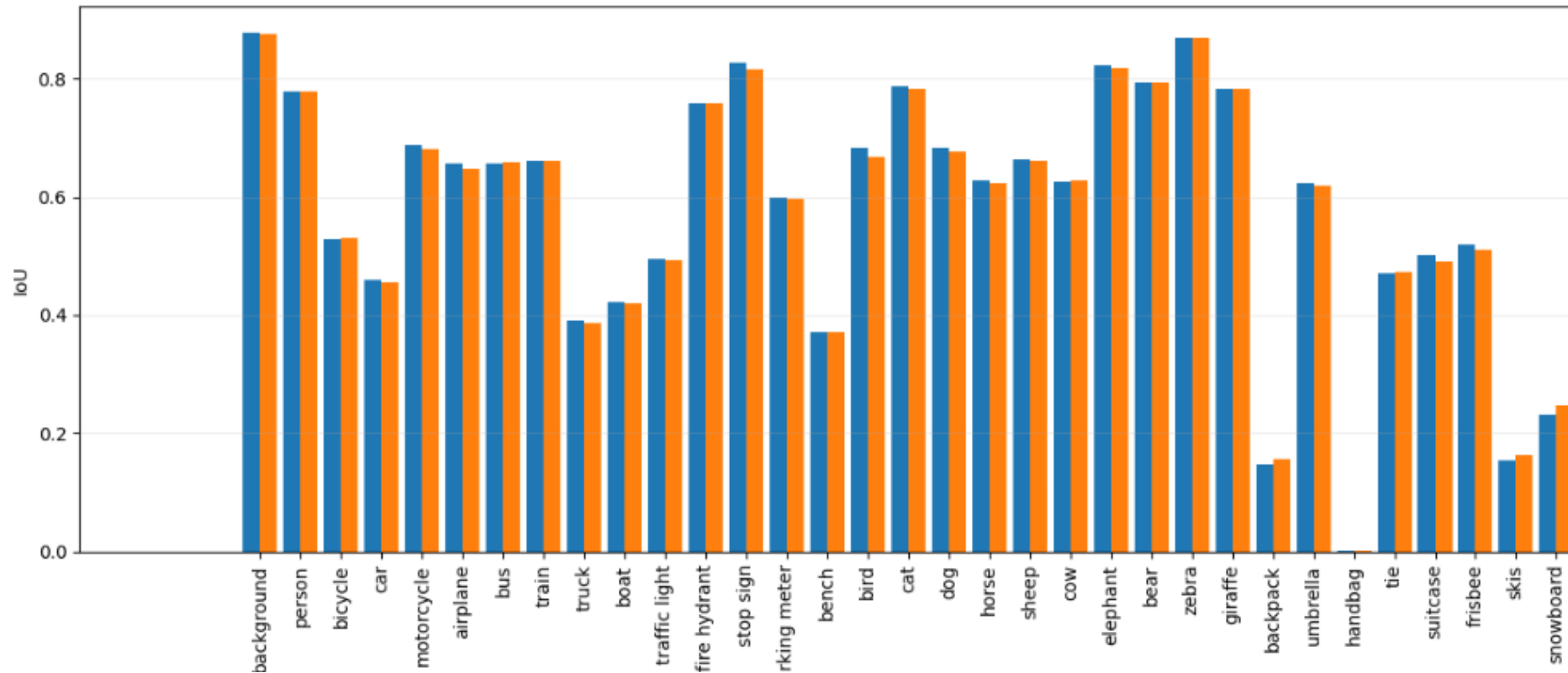
- Зануление весов в тензорах
- Слои не меняются
- Используем `torch.nn.utils.l1_unstructured`
- Pruning percent:
10/20/25/30/35/40/45/50/55/60%



Layer damage

	layer_name	n_params	amount=0.1	amount=0.2	amount=0.25	amount=0.3	amount=0.35	amount=0.4	amount=0.45	amount=0.5	amount=0.55	amount=0.6
0	decoder.blocks.0.conv1.0	7077888	0.004348	0.014722	-0.009513	-0.025666	-0.107908	-0.251138	-0.379416	-0.605476	-1.042825	-1.728848
1	decoder.blocks.1.conv1.0	884736	-0.008136	-0.019750	-0.038010	-0.068498	-0.073212	-0.113443	-0.185409	-0.296298	-0.458351	-0.643751
2	decoder.blocks.0.conv2.0	589824	-0.007007	-0.036827	-0.088763	-0.145936	-0.213388	-0.297239	-0.432321	-0.626457	-0.944829	-1.422232
3	encoder.layer4.0.conv2	2359296	-0.000438	-0.002486	-0.013441	-0.004920	-0.023377	-0.044331	-0.056636	-0.086114	-0.141031	-0.216207
4	encoder.layer4.1.conv2	2359296	0.001329	-0.009227	-0.010845	-0.012133	-0.017548	-0.037241	-0.059745	-0.126404	-0.211126	-0.423312
5	encoder.layer4.2.conv2	2359296	0.000584	0.000754	0.006029	0.006792	0.005633	0.013518	0.025201	0.036800	0.008968	-0.016558
6	encoder.layer4.0.downsample.0	2097152	0.004250	0.000396	-0.003898	-0.001866	-0.026730	-0.033313	-0.045162	-0.058290	-0.122982	-0.196594
7	encoder.layer4.0.conv3	1048576	-0.001973	-0.006577	-0.005245	-0.003201	-0.010538	-0.020340	-0.014082	-0.025955	-0.061157	-0.140697
8	encoder.layer4.1.conv1	1048576	-0.001219	-0.015080	-0.011089	-0.022230	-0.032321	-0.072765	-0.084507	-0.146139	-0.229785	-0.328168
9	encoder.layer4.1.conv3	1048576	0.001800	0.009242	0.017327	0.003383	0.003466	0.001276	-0.001740	-0.011218	-0.065312	-0.113395
10	encoder.layer4.2.conv1	1048576	0.002363	0.000086	-0.012279	-0.014579	-0.027573	-0.050256	-0.094056	-0.103965	-0.138548	-0.244355
11	encoder.layer4.2.conv3	1048576	0.001815	-0.011006	-0.023878	-0.048494	-0.077692	-0.125536	-0.134569	-0.137103	-0.203866	-0.353014
12	encoder.layer3.0.conv2	589824	0.002694	0.004146	-0.000575	0.005707	0.008026	0.018936	0.008196	-0.020382	-0.052199	-0.113443
13	encoder.layer3.1.conv2	589824	0.002667	0.004154	0.001669	0.004473	-0.000998	0.001234	-0.000706	-0.002363	-0.012931	-0.014108
14	encoder.layer3.2.conv2	589824	-0.002995	-0.001535	-0.003800	-0.001234	-0.011364	-0.001979	-0.011924	-0.017491	-0.030276	-0.043795
15	encoder.layer3.3.conv2	589824	-0.000253	0.000525	0.001550	0.000128	0.001323	0.003126	0.001904	0.007448	0.010645	0.010803

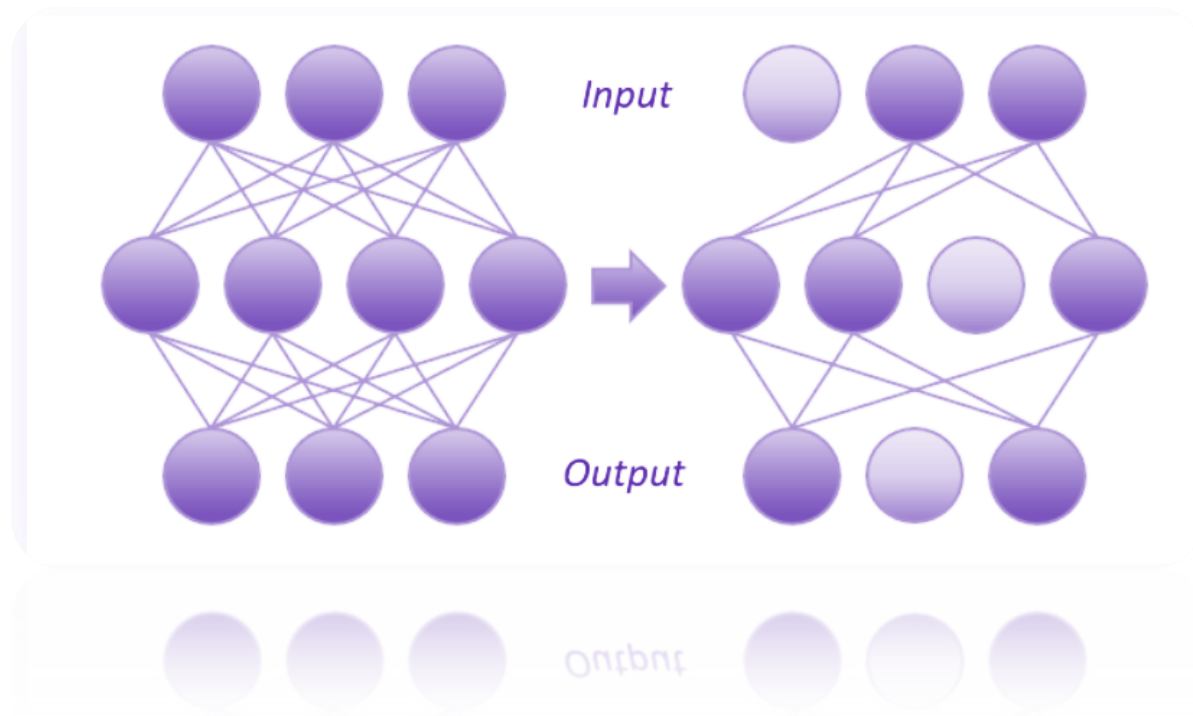
Main metric (mIoU)



mIoU baseline = 0.4819

mIoU unstructured pruning model = 0.4804

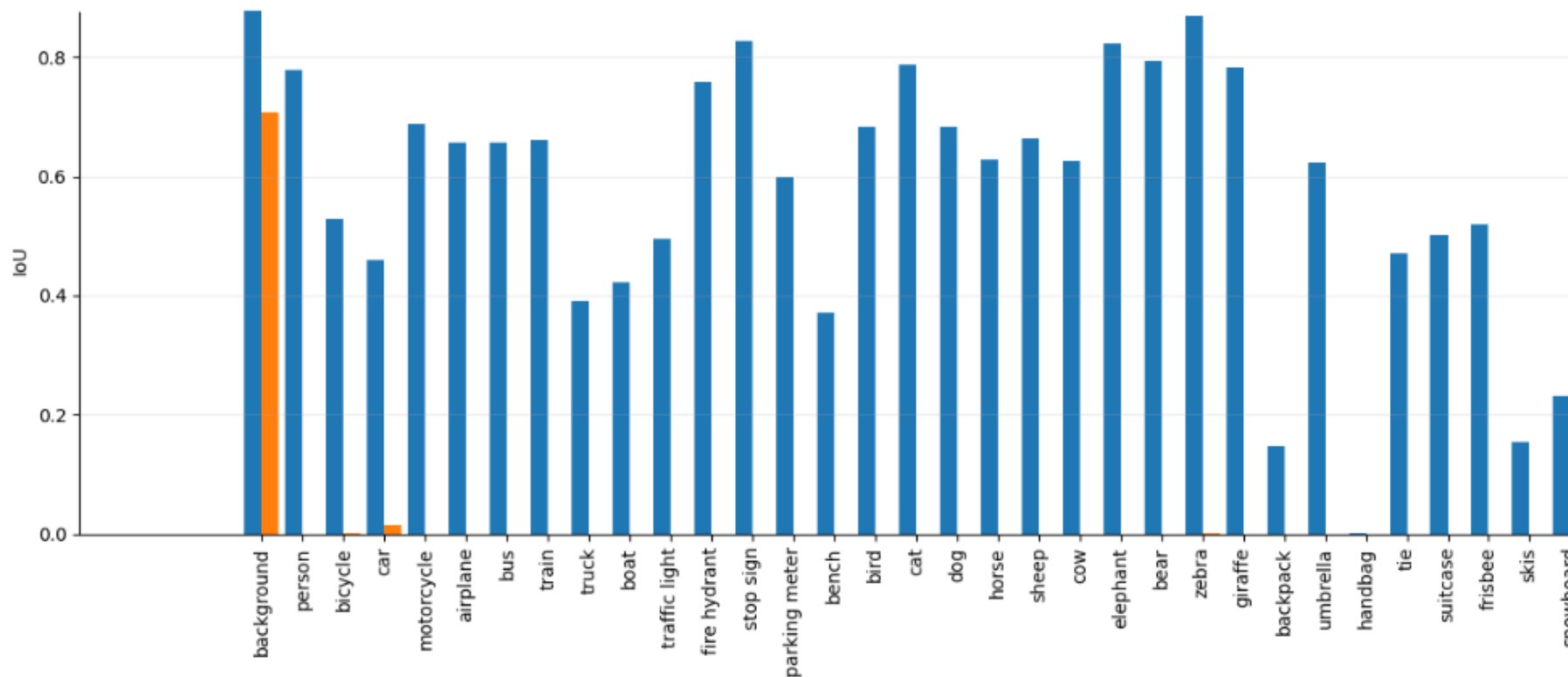
Structured Pruning



- Использовали torch-pruning
- Уменьшено 35.76% параметров:

51 524 833 -> 33 099 649

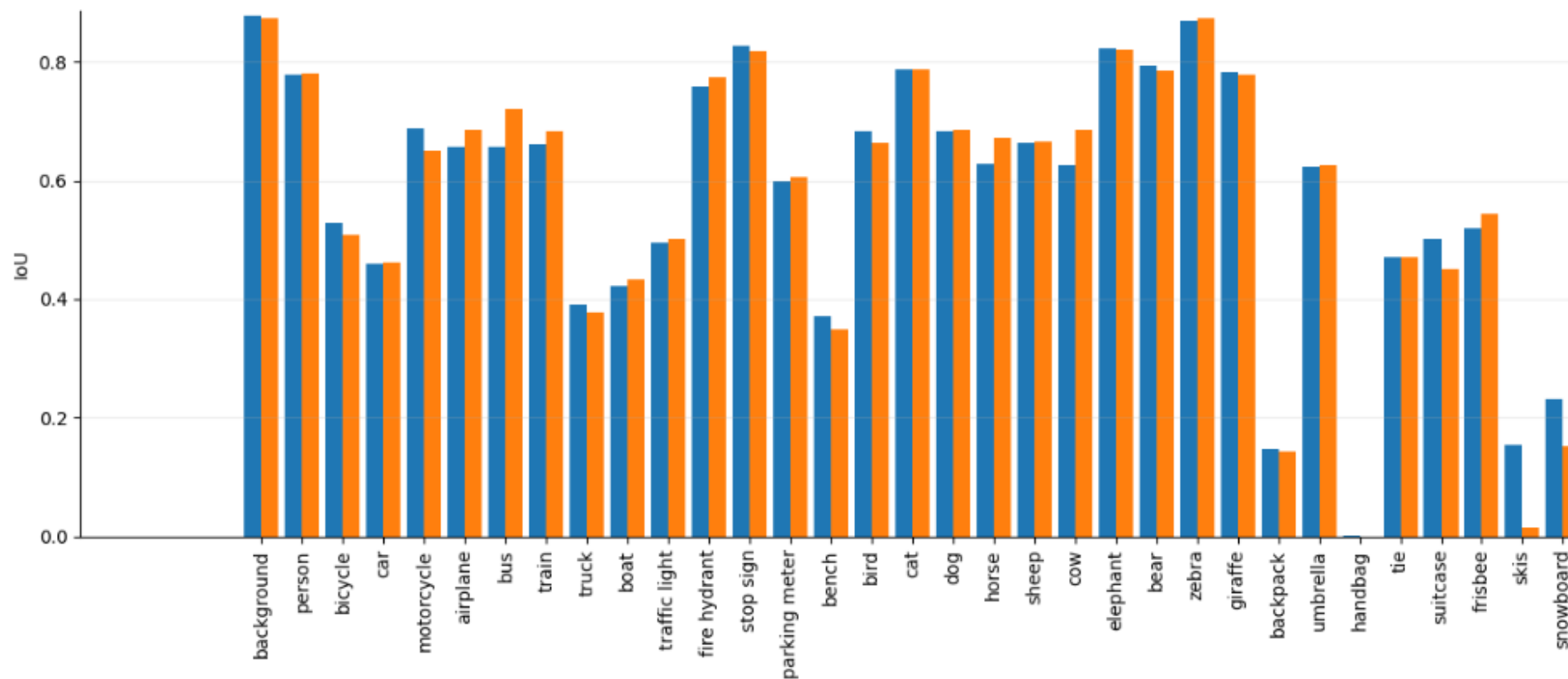
Main metric (mIoU)



mIoU baseline = 0.4819

mIoU structured pruning model = 0.01

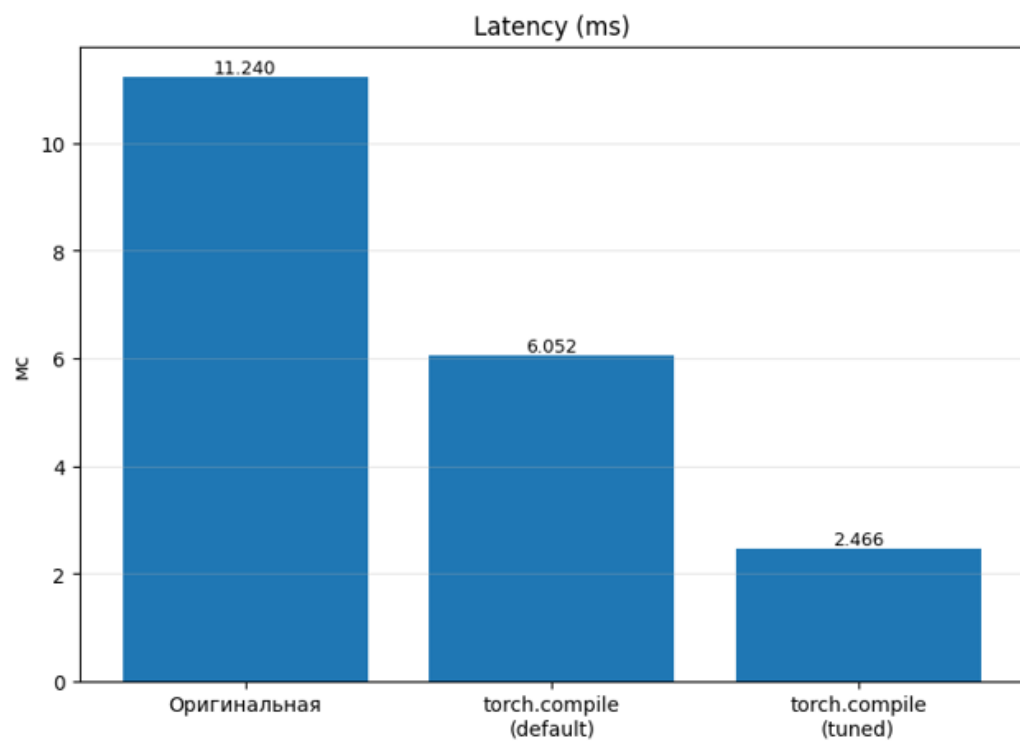
Main metric (mIoU) after new training



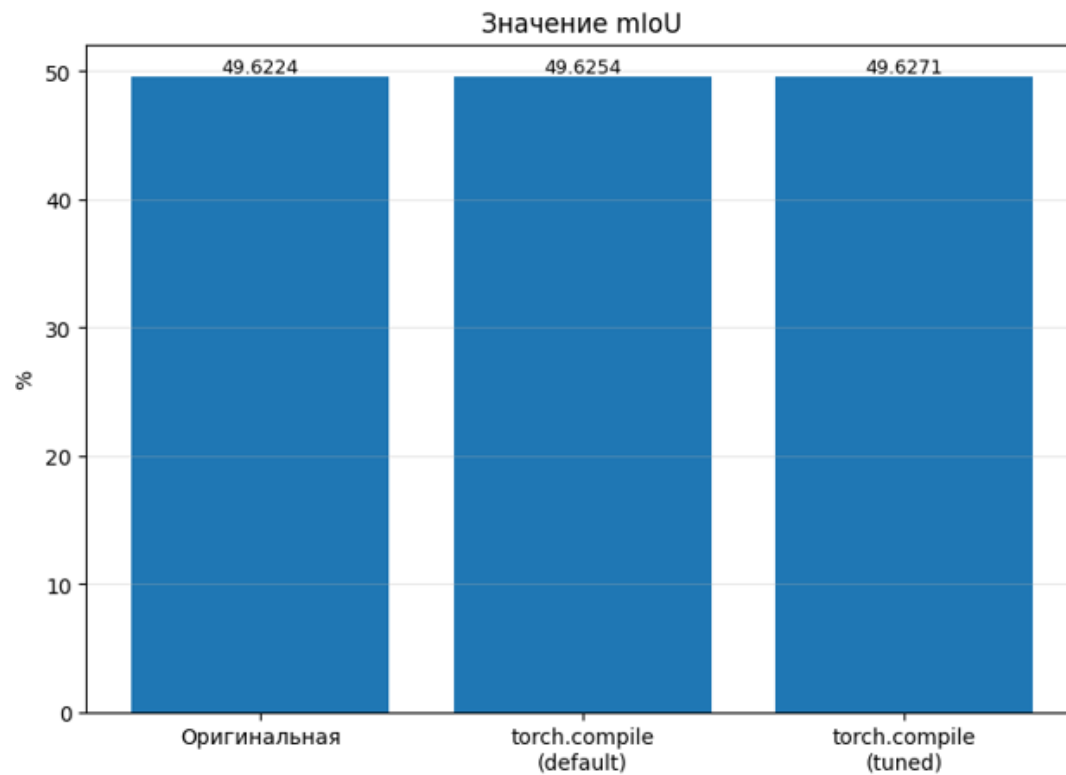
mIoU baseline = 0.4819

mIoU unstructured pruning model = 0.4608

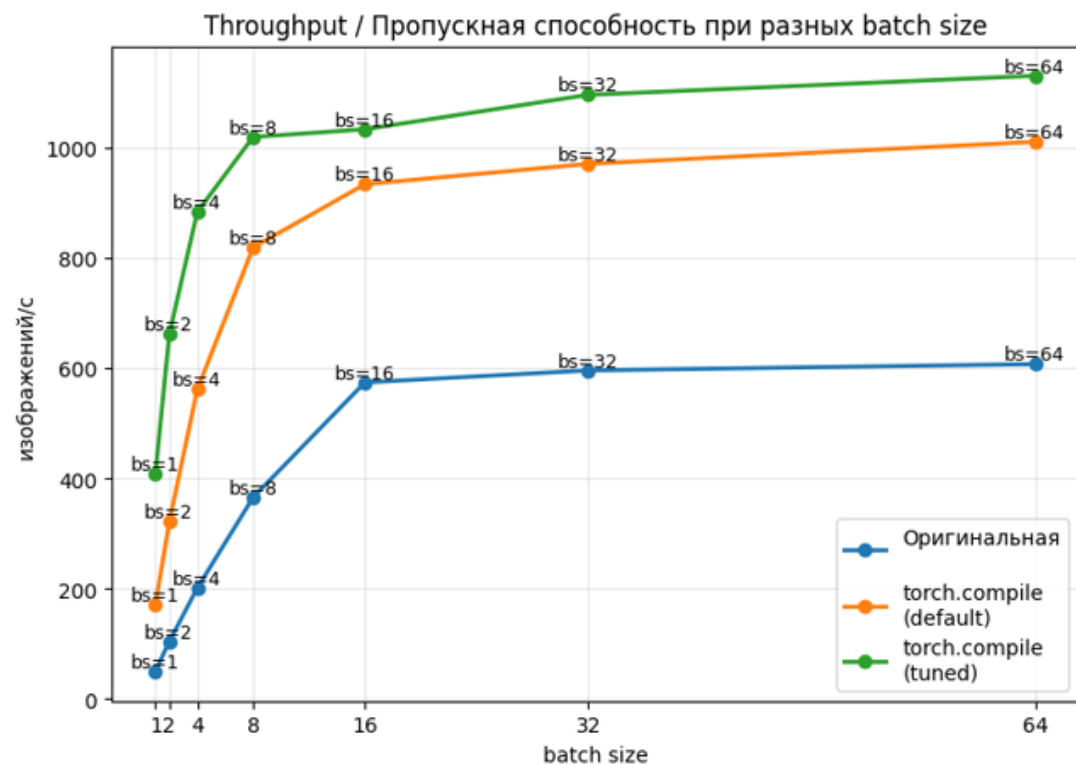
Latency



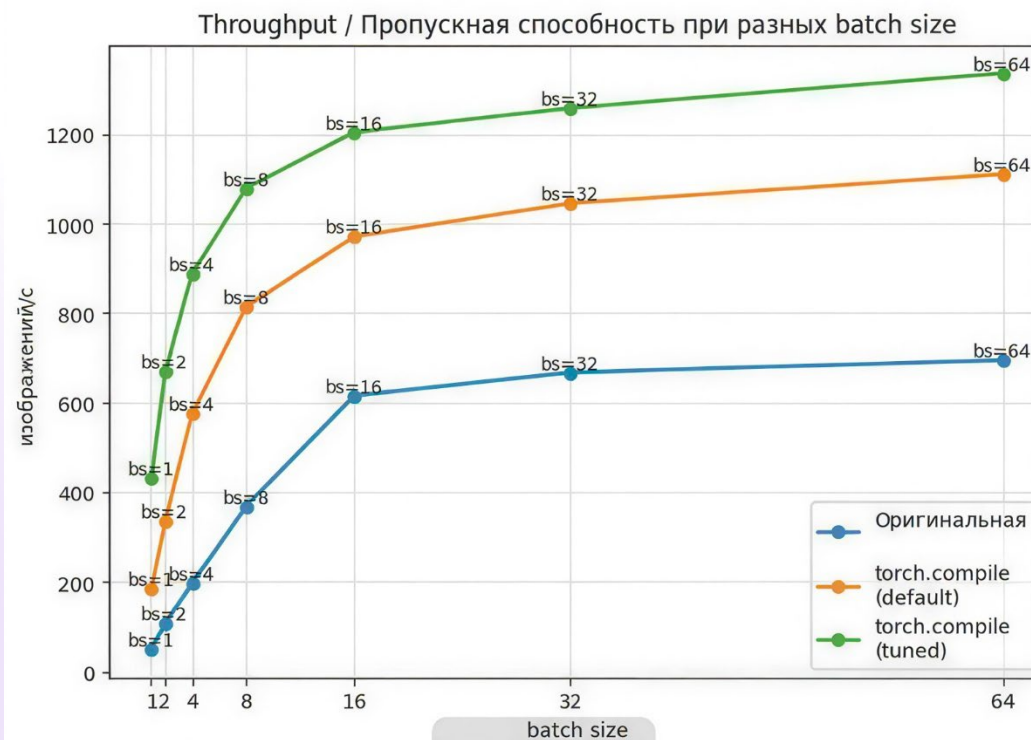
Main metric



Throughput



Throughput with pruning



2:4 semi structured pruning + finetuning

Не удалось достичь ускорения

Модель:

- почти полностью состоит из Conv2d
- не содержит тяжёлых Linear-слоёв

2:4 sparsity в PyTorch:

- ускоряется через sparse matmul (GEMM)
- эффективно работает для nn.Linear
- не даёт ускорения для Conv2d (т.к. Conv2d + 2:4 mask -> вычисляется как обычный dense conv с нулями)

Unstructured pruning

- При обнулении 20.20% весов сверточных слоёв модель сохраняет устойчивость и качество, что говорит о наличии параметрической избыточности.
- Однако, такой тип спарсификации не меняет архитектуру и в стандартном runtime не даёт заметного ускорения инференса.

Structured pruning

- Из 51 524 833 параметров было удалено 35.76% параметров. Осталось 33 099 649 параметров
- Такой подход уже меняет архитектуру и может дать реальное ускорение

Thanks for you attention!